

DEADLOCKS

CLASSICAL SYNCHRONISATION PROBLEM

```
void mutex_error_deadlock()
{
    mutex mut_a, mut_b;
    vector<thread> threads;

    auto thread_a = [&](int i)
    {
        mut_a.lock();
        cout << "Filling data" << " a"; // IO Wait
        mut_b.lock(); // May be locked by thread_b
        cout << "Transferring to" << " b" << endl;
        mut_b.unlock();
        mut_a.unlock();
    };
};
```

```
auto thread_b = [&](int i)
{
    mut_b.lock();
    cout << "Filling data " << " b"; // IO Wait
    mut_a.lock(); // May be locked by thread_a
    cout << "Transferring to " << "a" << endl;
    mut_a.unlock();
    mut_b.unlock();
};

threads.emplace_back(thread_a, 0);
threads.emplace_back(thread_b, 1);

for (auto& t : threads)
    t.join();
}
```

Asssuming namespace std

SUNSHINE SCENARIO...

```
auto thread_a = [&](int i) {  
    mut_a.lock();  
    cout << "Filling data" << " a"; // IO Wait  
    mut_b.lock();  
    cout << "Transferring to" << " b"; // IO Wait  
    mut_b.unlock();  
    mut_a.unlock(); };
```

```
auto thread_b = [&](int i) {  
    mut_b.lock();  
    cout << "Filling data" << " b"; // IO Wait  
    mut_a.lock();  
    cout << "transferring to" << " a"; // IO Wait  
    mut_a.unlock();  
    mut_b.unlock(); };
```

t	Thread A	Thread B
0	mut_a.lock(); (uncontended)	
1	std::cout << "Filling data" << " a";	
2	mut_b.lock(); (uncontended)	
3		mut_b.lock(); (contended - block)
4	std::cout << "Transferring to" << " b";	
5	mut_b.unlock();	
6		std::cout << "Filling data" << " b";
7		mut_a.lock(); (contended - block)
8	mut_a.unlock();	
9		std::cout << "Transferring to" << " a";



RAINY DAY SCENARIO...

```
auto thread_a = [&](int i) {
    mut_a.lock();
    cout << "Filling data" << " a"; // IO Wait
    mut_b.lock();
    cout << "Transferring to" << " b"; // IO Wait
    mut_b.unlock();
    mut_a.unlock(); };
```

```
auto thread_b = [&](int i) {
    mut_b.lock();
    cout << "Filling data" << " b"; // IO Wait
    mut_a.lock();
    cout << "transferring to" << " a"; // IO Wait
    mut_a.unlock();
    mut_b.unlock(); };
```

t	Thread A	Thread B
0	mut_a.lock(); (uncontended)	
1		mut_b.lock(); (uncontended)
2	std::cout << "Filling data" << " a";	
3		std::cout << "Filling data" << " b";
4	mut_b.lock(); (contended - block)	
5		mut_a.lock(); (contended - block)
6	Deadlocked!	

DEADLOCK CHARACTERISATION

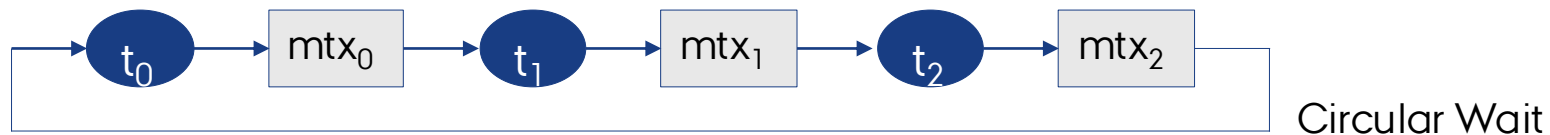
Deadlock WILL occur if ALL the following four (necessary) conditions are available:

Mutual exclusion. At least one resource is held by a single thread at a time. Other requests to the resource is delayed, until the resource is released

Hold and wait. A thread must hold at least one resource and wait for another, currently held by another thread.

No preemption. Only the thread holding a resource can also release it!

Circular wait. Threads must be waiting for resources in round-robin fashion



PROGRAM ANALYSIS

```
auto thread_a = [&](int i) {  
    mut_a.lock();  
    cout << "Filling data" << " a"; // IO Wait  
    mut_b.lock();  
    cout << "Transferring to" << " b"; // IO Wait  
    mut_b.unlock();  
    mut_a.unlock(); };
```

```
auto thread_b = [&](int i) {  
    mut_b.lock();  
    cout << "Filling data" << " b"; // IO Wait  
    mut_a.lock();  
    cout << "transferring to" << " a"; // IO Wait  
    mut_a.unlock();  
    mut_b.unlock(); };
```

Mutual exclusion. Thread_a locks mut_a and thread_b locks mut_b. A mutex can only be locked once, so other requests will wait.



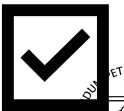
Hold and wait. Thread_a can hold mut_a and wait for mut_b, potentially held by thread_b.



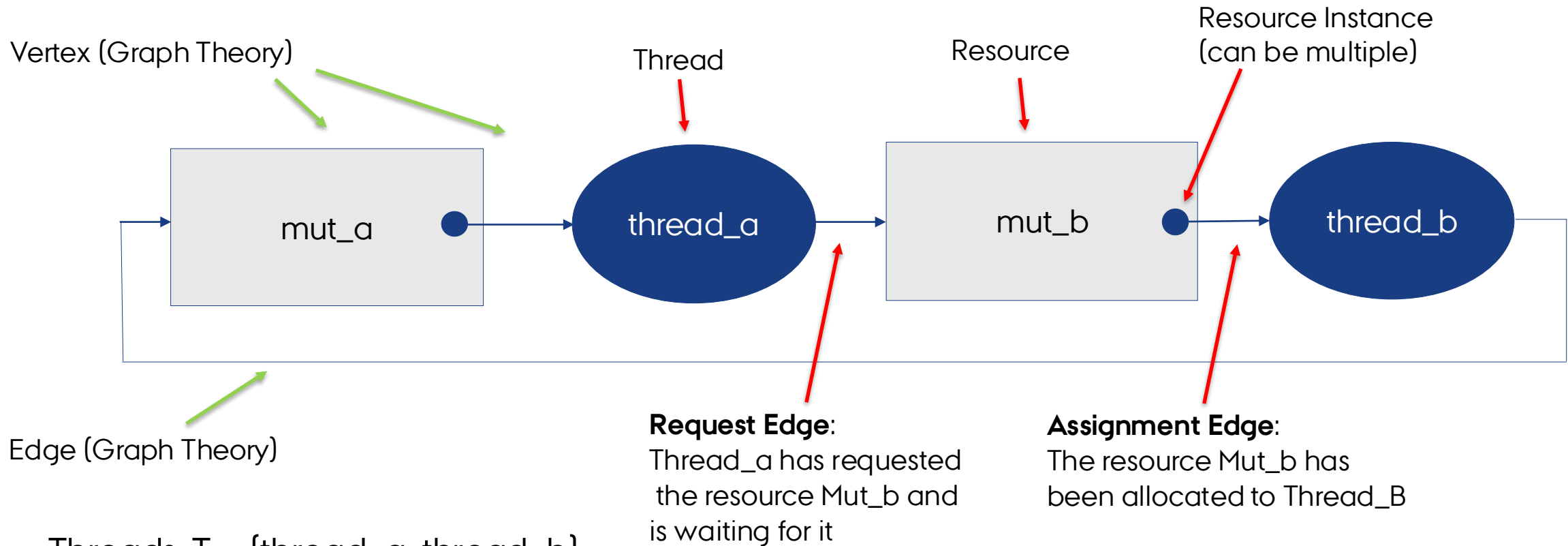
No preemption. A Mutex can only be released by the thread that locked it.



Circular wait. Thread_a locks mut_a and waits for mut_b, thread_b locks mut_b and waits for mut_a



RESOURCE-ALLOCATION GRAPH (RAG)



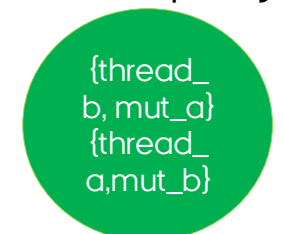
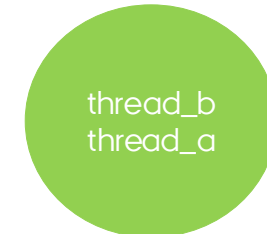
Threads, $T = \{\text{thread_a}, \text{thread_b}\}$

Resources, $R = \{\text{mut_a}, \text{mut_b}\}$

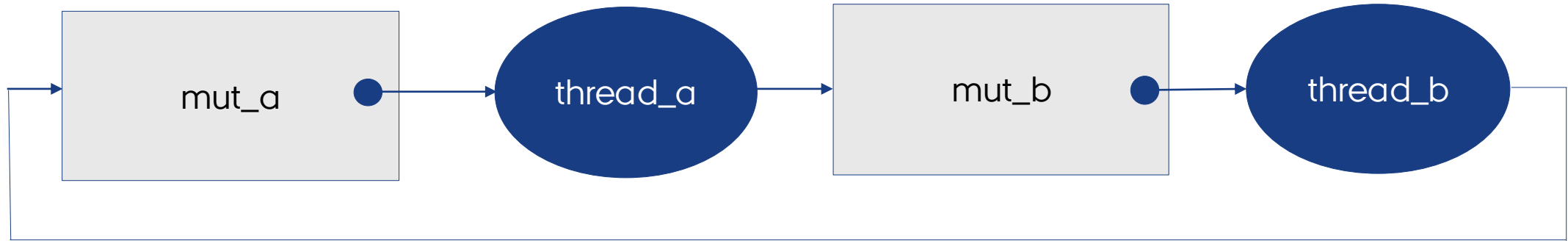
Request Edges: $\{\{\text{thread_a}, \text{mut_b}\}, \{\text{thread_b}, \text{mut_a}\}\}$

Assignment Edges: $\{\{\text{mut_a}, \text{thread_a}\}, \{\text{mut_b}, \text{thread_b}\}\}$

Set of threads Set of req. edges



RESOURCE-ALLOCATION GRAPH



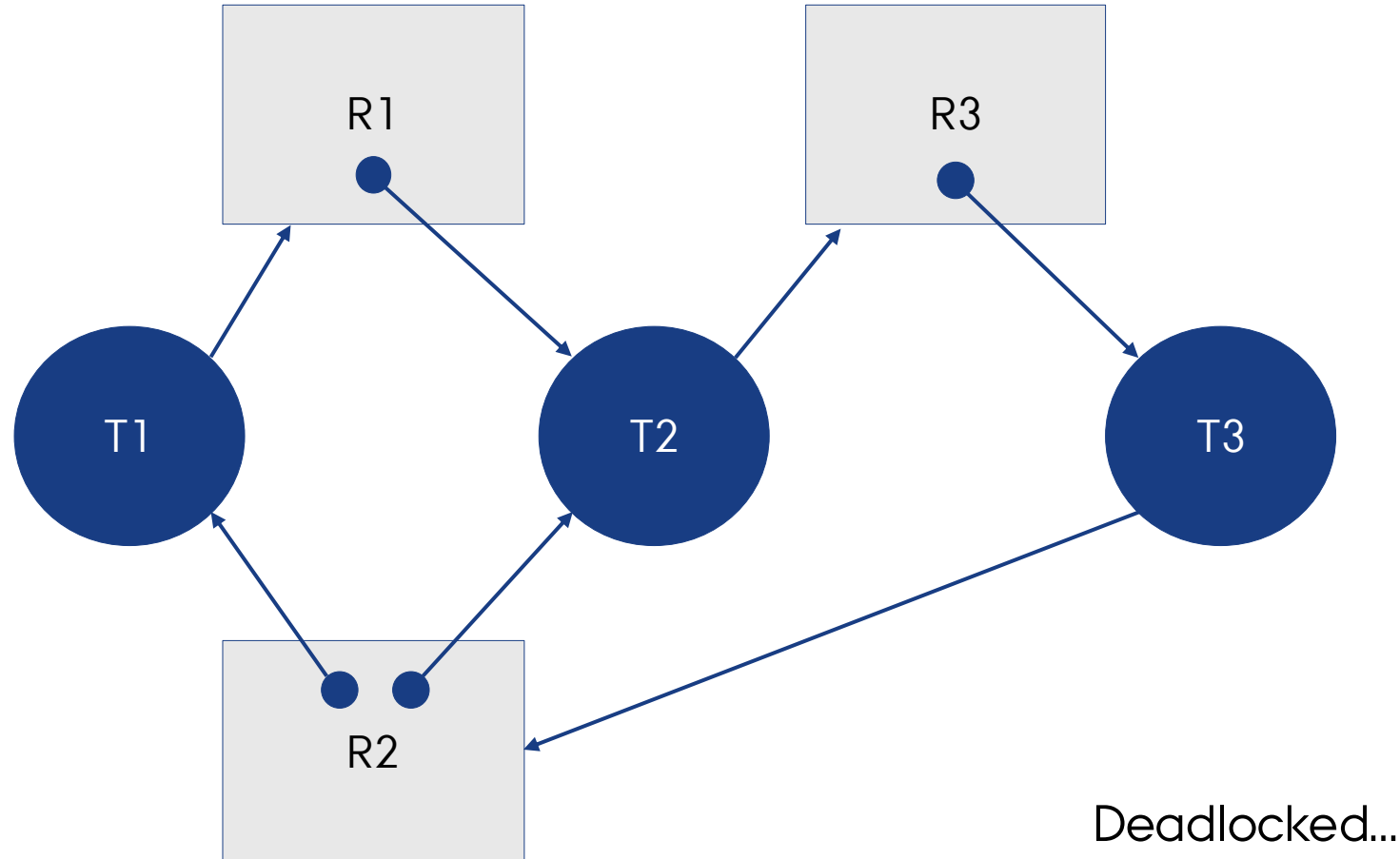
If RAG has **NO cycles**, then **NO** threads in the systems are **deadlocked**!

If the RAG has a cycle, then a deadlock **MAY** exist!

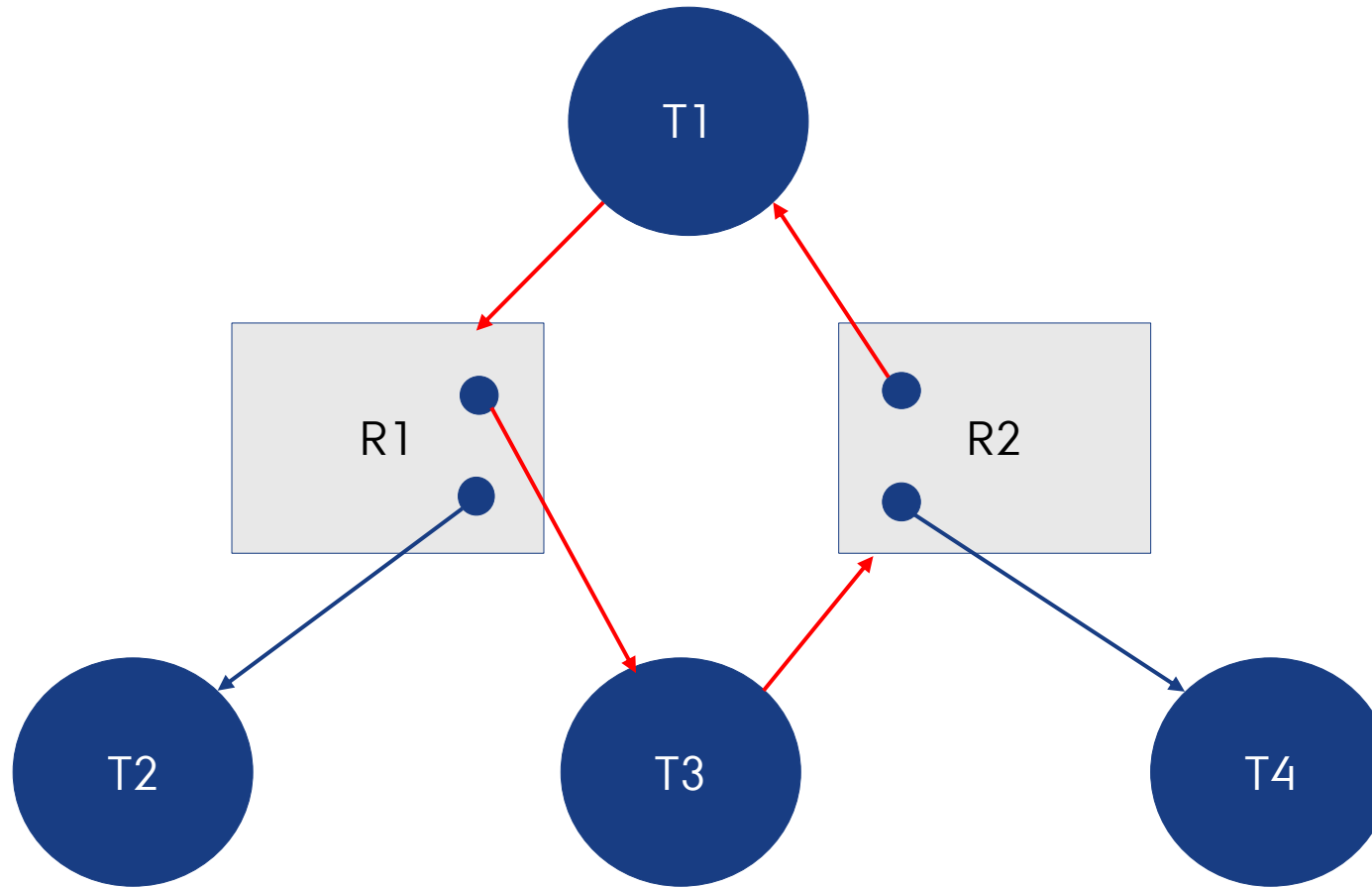
If each resource in the cycle has only **ONE instance**, then the conditions for a deadlock are satisfied and a deadlock has occurred!

If each resource in the cycle has **SEVERAL instances** (ex. counting semaphore), then conditions for a deadlock are available, but not satisfied, thus there is no guarantee of deadlock

MULTIPLE RESOURCE INSTANCES



MULTIPLE RESOURCE INSTANCES



Deadlocked... NOT!

HANDLING DEADLOCKS

Three (four) ways to handle deadlocks:

Ignore them: Do nothing, maybe it's ok!?!

Deadlock Prevention: Prevent they can ever occur!

Deadlock avoidance: Provide information to OS, to let it manage resources in a safe way

(Deadlock detection and recovery: If no prevention or avoidance, detect and mitigate deadlocks if/when they occur)

DEADLOCK PREVENTION 1:2

Avoid one of the four necessary conditions, can be satisfied!

Mutual Exclusion. Shareable resources, such as read-only files or immutable data types, do not require atomic access and therefore cannot be involved in a deadlock

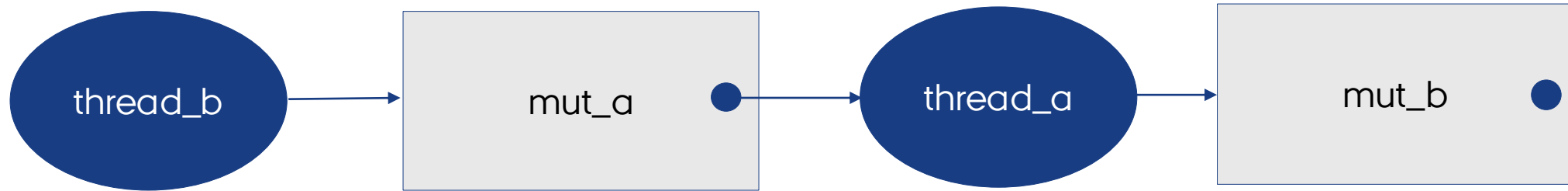
Hold and wait. A thread must not hold a resource, when requesting another. Can be avoided by allocating all resources before execution. Alternatively, any resource must be released, before requesting another. This does however not eliminate starvation

DEADLOCK PREVENTION 2:2

No preemption. If a thread holds a resource and cannot immediately obtain another resource, then the resource must be preempted (released), and the thread can only resume when both are available. Alternatively, preempt another waiting thread for missing resource

Circular Wait. Ensure a total ordering/hierarchy of all resource types and require threads request resources in an increasing order of enumeration. *This is something we can do!*

LOCK ORDERING/HIERARCHY



Both threads request first mut_a, then mut_b → No deadlock!

```
auto thread_a = [&](int i) {  
    mut_a.lock();  
    cout << "Filling data" << " a"; // IO Wait  
    mut_b.lock();  
    cout << "Transferring to" << " b"; // IO Wait  
    mut_b.unlock();  
    mut_a.unlock(); };
```

```
auto thread_b = [&](int i) {  
    mut_a.lock();  
    cout << "Filling data" << " b"; // IO Wait  
    mut_b.lock();  
    cout << "transferring to" << " a"; // IO Wait  
    mut_b.unlock();  
    mut_a.unlock(); };
```

DEADLOCK AVOIDANCE

Run-time deadlock prevention

The system monitors how resources are requested by the running code and uses a deadlock-avoidance algorithm to ensure circular-wait condition cannot happen.

This is enforced inside the Linux kernel by the *lockdep* tool.

Linux cannot enforce lock-ordering for a user space application. The deadlock avoidance-algorithm must be part of the applications's logic.

Algorithms such as Bankers algorithm can be employed.

DEADLOCK DETECTION

Avoiding deadlocks can be resource-costly

Instead of deadlock-prevention or avoidance algorithm, the system may employ an algorithm that examines the system occasionally to detect and recover from deadlocks:

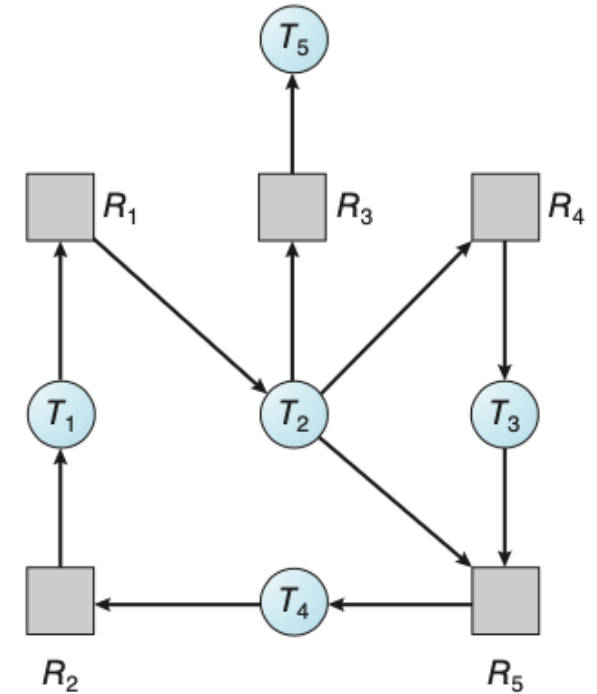
Finding circular wait conditions by searching for loops in the system wait-for graph. ($O(N^2)$)

Use an algorithm to break the loop:

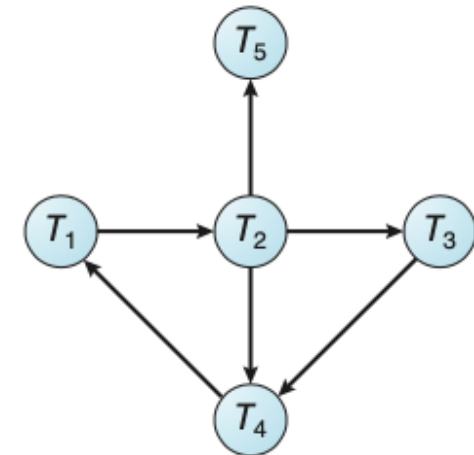
Abort thread

Preempt resource

Resource Allocation Graph



Wait-for Graph





AARHUS
UNIVERSITY